

量子クラウド環境を想定した**MIP**による 回路分割最適化とエラー抑制効果の検証

第203回HPC・第17回QS合同研究発表会

2026/3/17

森俊夫, 東野仁政, 枡本尚之,
宮永崇史, 宮地孝輔, 藤田大樹
大阪大学 QIQBクラウドチーム

目次

- 本研究開発の目的
- 量子回路切断と量子エラー抑制とは
- 提案手法
- 評価実験
- クラウドサービスへの適用と今後の課題

本研究開発の目的

量子回路切断機能を有するサービスは存在しない！

- つまり、今リリースすれば世界初！！
- 量子回路切断とは、大きな量子回路を切断して小さな回路に分割して複数回実行して結果を復元する機能である。
- 手法によりますが基本スケールしないため、サービスとしての使い所が難しい。

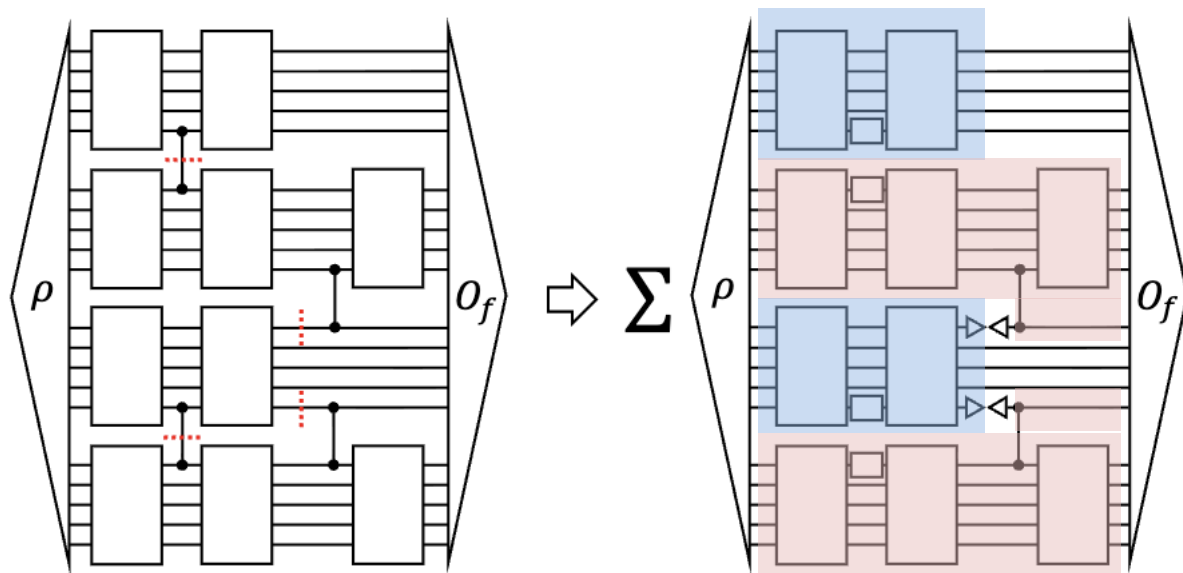
量子回路切断機能を有するサービスは存在しない！

- 量子コンピュータ実機の現状を考慮して、使えそうな所に落とし込んでみたのが今回の成果
- 具体的には、量子回路切断を量子エラー抑制として使用する。

量子回路切断と量子エラー抑制とは

量子回路切断とは

量子回路を切断して複数の小さな部分回路に分割し、古典後処理によって元の結果を再構成する技術である。[1], [2]



[1] Mitarai, Kosuke and Fujii, Keisuke: Constructing a virtual two-qubit gate by sampling single-qubit operations, New Journal of Physics, Vol. 23, No. 2(2021), pp. 23021.

[2] Mitarai, Kosuke and Fujii, Keisuke: Overhead for simulating a non-local channel with local channels by quasiprobability sampling, Quantum, Vol. 5 (2021), pp. 388.

量子回路切断の手法

代表的な切断手法として Gate Cutting [1] と Wire Cutting [2] がある。

手法	Gate Cutting	Wire Cutting
基本概念	2量子ビットゲートを切断	経路（wire）を切断
オーバーヘッド	比較的低い	比較的高い
サンプリングコスト係数 γ^2	4	16
総ショット数増加率（ k 箇所切断時）	4^k	16^k
実装の容易さ	容易	やや複雑（中間測定が必要）
メリット	・ 特定のエラー源（2Qゲート）を除去しやすい。 ・ サンプリングコストが比較的低く抑えられる。	・ 柔軟性が高い（どこでも切れる）。 ・ 量子ビットの物理的な移動（テレポーテーション的）を表現できる。
デメリット	・ ゲートがない場所では切れない。 ・ 分解後のパリティ計算で、状態によっては期待値が消失する問題がある。	・ 項数（分解の項）が多くなりがちで、実行回数（ショット数）が爆発しやすい。 ・ 古典後処理の計算量が大きくなる傾向がある。

[1] Peng, Tianyi, Harrow, Aram, Ozols, Maris and Wu, Xiaodi: Simulating large quantum circuits on a small quantum computer, arXiv preprint arXiv:1904.00102 (2019).

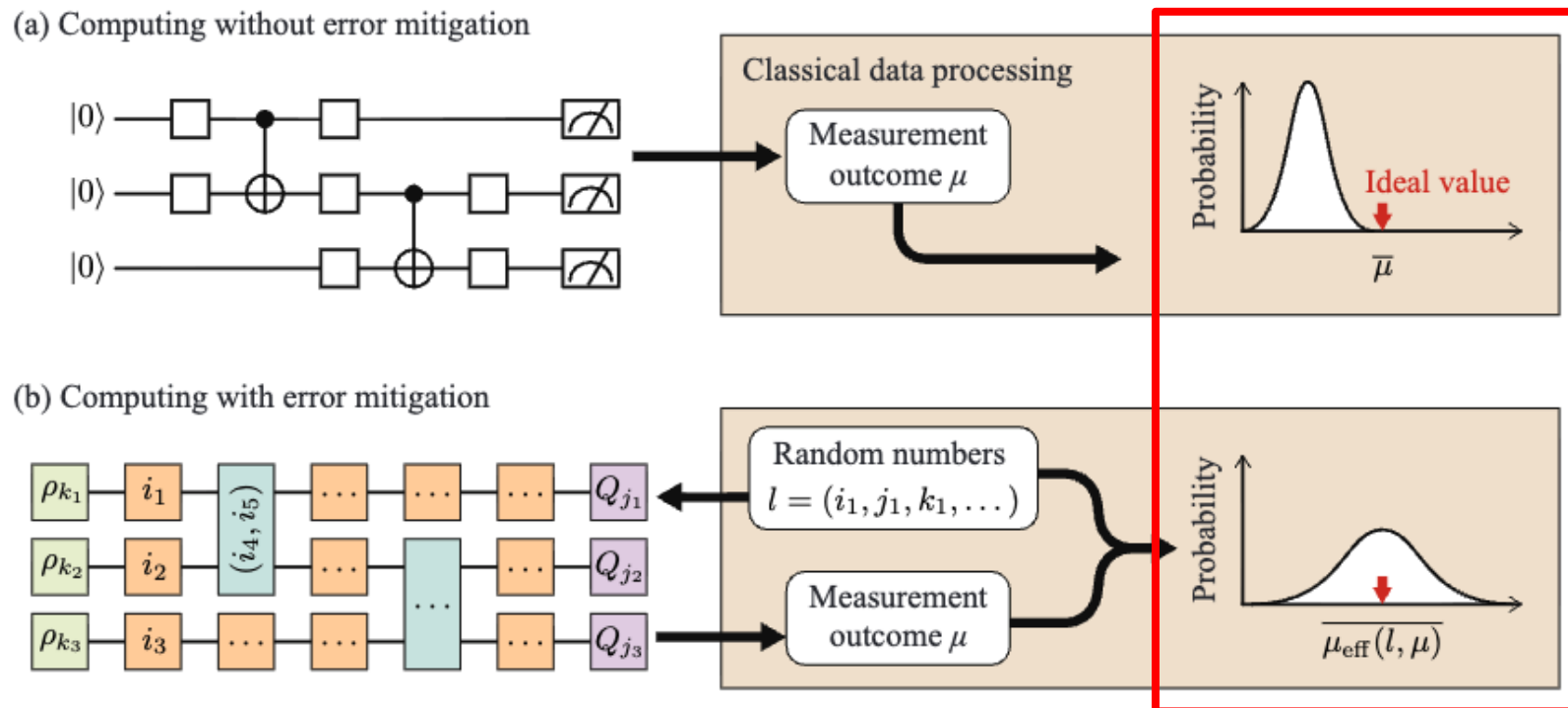
[2] Ren, Xiangyu, Zhang, Mengyu and Barbalace, Antonio: A hardware-aware gate cutting framework for practical quantum circuit knitting, Proceedings of the 43rd IEEE/ACM International Conference on Computer-Aided Design (2024), pp. 1-9.

量子回路切断のソフトウェア

名前	概要	URL
Qiskit circuit cutting	Qiskitのアドオンとして実装されている。回路切断の手法を実行時に選択可能。	https://github.com/Qiskit/qiskit-addon-cutting
PennyLane quantum circuit cutting	PennyLaneの機能として実装されており、切断位置を宣言的に指定可能。	https://pennylane.ai/qml/demos/tutorial_quantum_circuit_cutting
CutQC	MIPソルバで切断数を最小化するように最適化可能。	https://github.com/weiT1993/CutQC

量子エラー抑制とは

現状の量子コンピュータはまだノイズが多く、測定結果や計算結果に含まれるエラーを事後的に“取り除く”または“軽減する”手法である。[1], [2]



[1] Endo, Suguru, Benjamin, Simon C and Li, Ying: Practical quantum error mitigation for near-future applications, Physical Review X, Vol. 8, No. 3 (2018), pp. 31027.

[2] Endo, Suguru, Cai, Zhenyu, Benjamin, Simon C and Yuan, Xiao: Hybrid quantum-classical algorithms and quantum error mitigation, Journal of the Physical Society of Japan, Vol. 90, No. 3 (2021), pp. 32001.

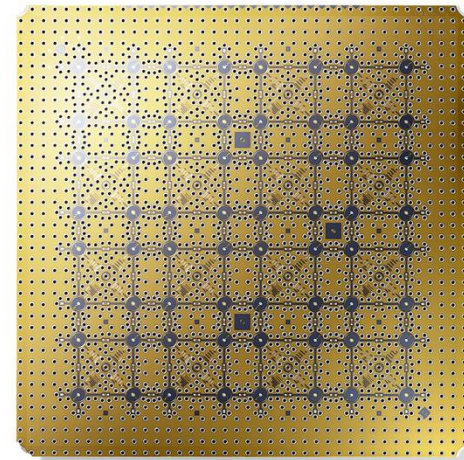
現状の環境

実行環境

本研究で想定している環境は、理化学研究所で研究開発された超伝導型の64量子ビットの量子コンピュータである。

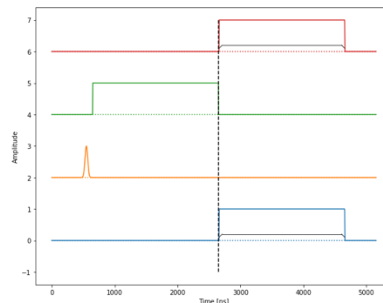
量子ビットは8x8の格子状に配置されてる。

量子プログラムは、マイクロ波に変換されて実行されるが、量子チップで実行される前後に古典コンピュータによる前処理、後処理が必要となる。



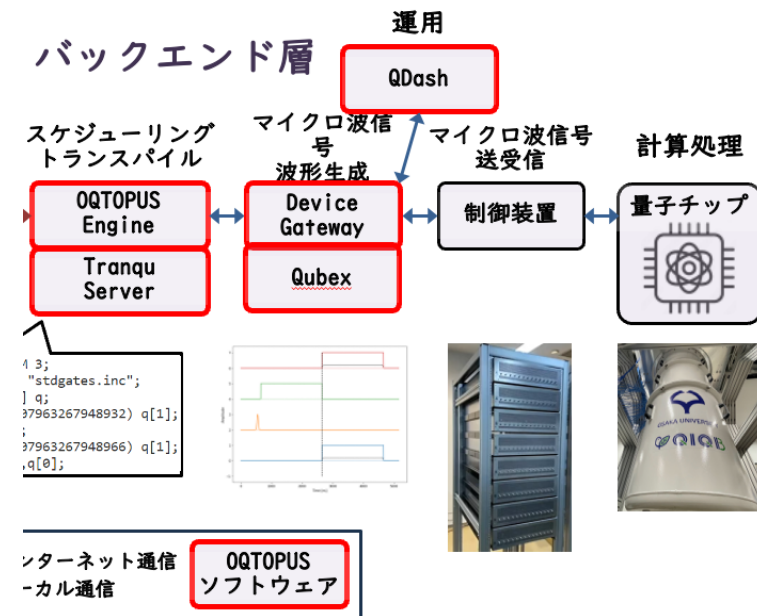
```
OPENQASM 3;
include "stdgates.inc";
qubit[2] q;
rz(1.5707963267948932) q[1];
sx q[1];
rz(1.5707963267948966) q[1];
cx q[1],q[0];
```

変換



量子プログラム (QASM)

パルス波形 (マイクロ波)



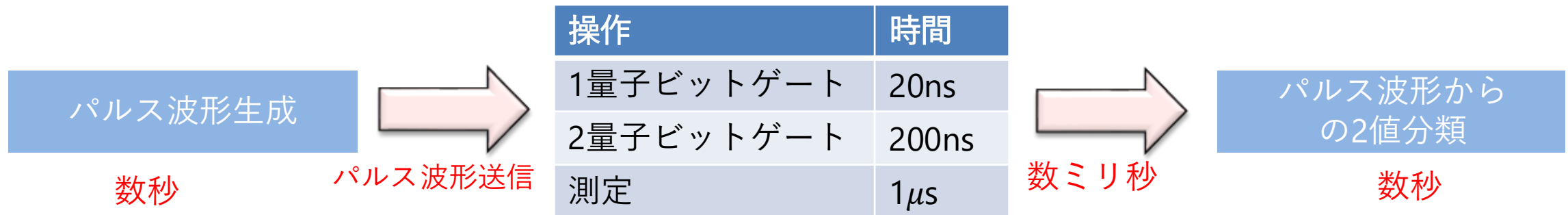
実行時間の内訳

量子コンピュータを制御装置を含む古典コンピュータと量子ビットを含む量子デバイスに分けた場合、実際に量子回路を実行する上での実行速度の割合は以下のようになる。

古典コンピュータでの前処理

量子デバイス上での実行

古典コンピュータでの後処理



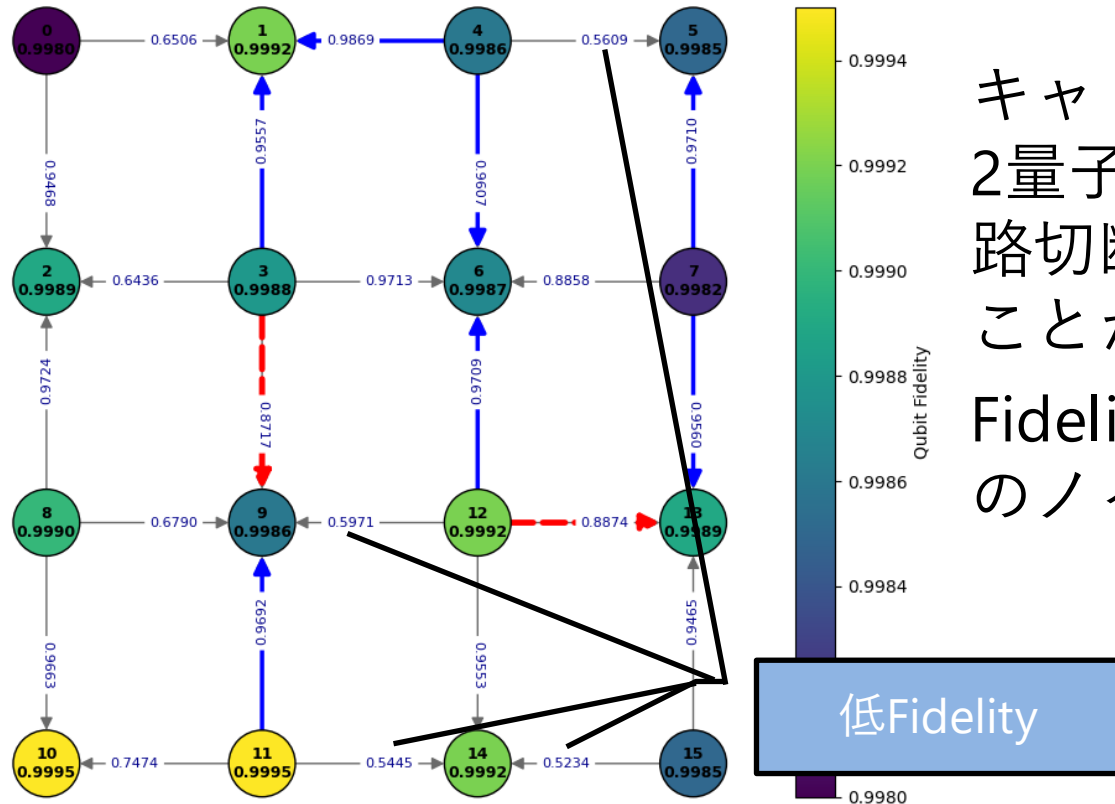
NISQデバイス（ノイズがある量子コンピュータ）実行可能な処理はせいぜい数十ゲート処理のため1ショットあたり、**数 μ s**の実行時間となる。

数千ショット実行したすると**数ミリ秒**となる。

- ・ 総ショット数増加率が増えると量子デバイスで実行される全体のジョブが増える。
- ・ 分解後の復元に必要なショット数は量子デバイス上での実行に効いてくる。
- ・ ショット数が増える事は、全体のジョブが増える事に比べれば比重は軽い。

キャリブレーション

超伝導型量子計算機は、定期的にキャリブレーションが行われ、量子ビットの周波数探索やゲートパラメータの最適化が実施される。キャリブレーションとは、量子ビットの共鳴周波数の特定から始まり、どのようなパルス形状や強度でゲート操作を行うかを決定するプロセスである。



キャリブレーションにより、各量子ビットおよび2量子ビットゲートのFidelityが変動する。量子回路切断を用いてノイズの大きいゲートを回避することが有効となる。

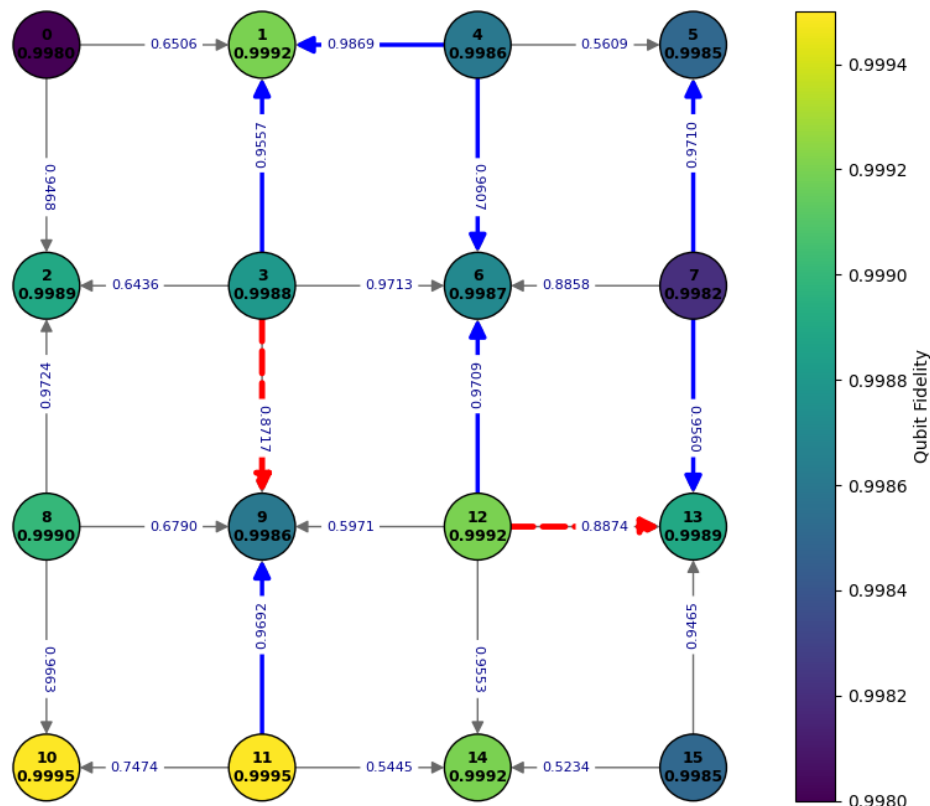
Fidelityが低いゲートを切断することで、回路全体のノイズを低減できる可能性がある。

提案手法

切断イメージ

4 x 4の格子状に配置された16量子ビットの量子回路に対して、GHZ状態を生成する10量子ビット回路を適用した場合の例である。

Transpiled Circuit (Blue=Used, Red=Cut)
Qubit: 16, Coupling: 24



GHZ状態とは、複数の量子ビット（3つ以上）が特殊な方法で「もつれ合った（エンタングルした）」状態のことで、すべての量子ビットが同時に「0」の状態と、すべての量子ビットが同時に「1」の状態の2つが、等しい確率で重ね合わさっている状態を指す。

■ 凡例

- ・ ノードが量子ビット
- ・ エッジが2量子ビットゲート
- ・ ノード内の数字は量子ビットのインデックス
- ・ ノード内の小数はFidelity（忠実度）
- ・ 青い線が結合された2量子ビットゲート
- ・ 赤い線が切断されたゲート

Gate Cuttingを使用する

CXゲートはパウリ演算子を用いて以下のように分解できる。

$$CX = \frac{1}{2}(I \otimes I + Z \otimes I + I \otimes X - Z \otimes X)$$

この分解では4つの項のみを使用し、係数の絶対値和は2となる。したがって、サンプリングコストの増加係数は $\gamma^2 = 4$ であり、**Wire Cutting**と比較して指数関数的に有利である。

k 箇所のゲートを切断する場合の総コストは 4^k で済む。

ジョブ数

1箇所の場合: $4^1 = 4$ 、2箇所の場合: $4^2 = 16$

3箇所の場合: $4^3 = 64$

但し、無限回数ショットが取れば復元できるが実機では無理なので有限ショット数で近似的に復元することとなる。

切断位置の最適化

効果的なエラー抑制を実現するためには、回路内のどのゲートを切断すべきかを適切に決定する必要がある。

量子回路をグラフとして扱い、混合整数計画法（MIP）を用いた推定を行った。

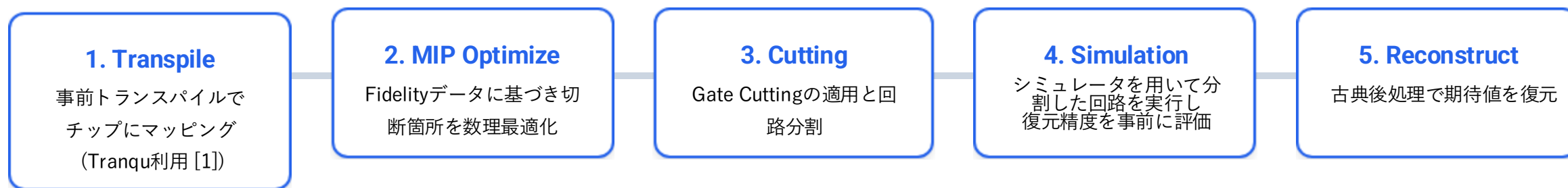
- ・ 量子回路を量子ビットをノード、2量子ビットゲートをエッジとする
- ・ 各エッジには、Fidelity（忠実度）を重みとして付与する

■MIPソルバの目的関数

1. 最大切断数制約: 計算コストの爆発を防ぐため、切断総数の上限を設定
2. Fidelityに基づく選択: Fidelityが低い（エラー率が高い）エッジの切断を優先し、かつ全体の切断数が最小となる解を探索
3. バランス制約: 切断による部分回路のfidelityが均等になるように、各部分回路のサイズを制御

提案手法のフロー

以下のステップで処理を行う



※今回の提案手法では事前に評価シミュレーションを行うことで、復元に失敗する可能性を低減しているが、そのためシミュレーション可能な回路サイズに制限される。

Stim [2] 等のクリフォード回路シミュレータを用いることで、より大規模な回路に対しても事前評価を行うことが可能である。その場合、クリフォード回路に限定されるため、回転ゲートをS等のクリフォードゲートに置き換える必要がある。

[1] QIQB: Tranqu (2025), (<https://tranqu.readthedocs.io/stable/>).

[2] Gidney, Craig: Stim (2025), (<https://github.com/quantumlib/Stim>).

評価実験

評価方法（1/2）

提案手法の有効性を検証するため、16量子ビット（4x4格子）の超伝導量子ビットシステムを想定したシミュレーション評価を行った。

理想環境下で各手法の基本的な性能差を明確に把握し、その後ノイズ環境下での実用的な効果を検証するため、2段階の評価を行った。

■ 2段階評価

- 第1段階: ノイズのない理想環境下で、Gate CuttingとWire Cuttingのサンプリングオーバーヘッドと復元精度を比較
- 第2段階: キャリブレーションデータに基づくノイズモデルを用いて、Gate Cuttingのエラー抑制効果を評価

評価方法 (2/2)

■第1段階評価: サンプリングオーバーヘッドと復元精度の比較

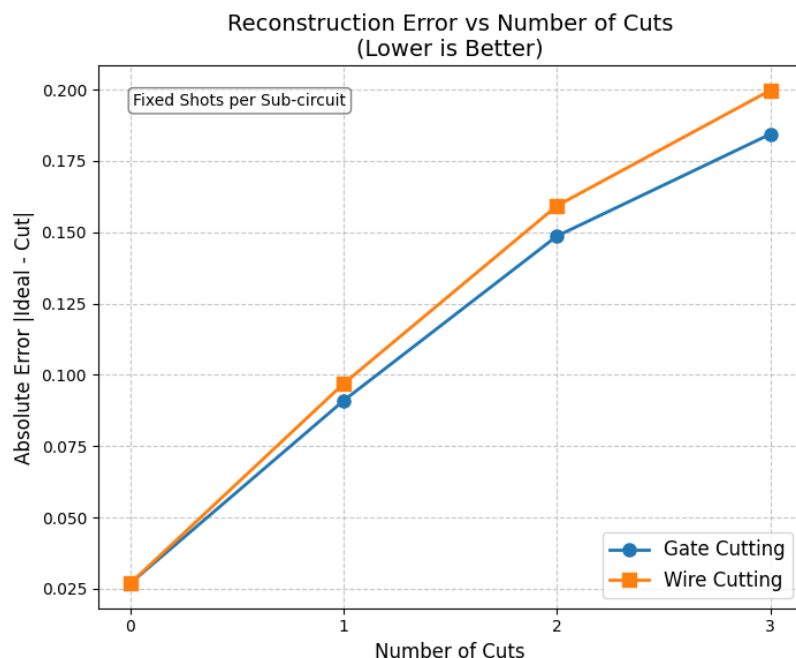
- 回路: ユニバーサルランダム回路 (8量子ビット回路、深さ4)
- チップ: 物理チップは想定せず、完全結合を仮定
- 切断数: 3箇所まで
- 実行ショット数: 各部分回路2000ショット固定
- シミュレータ: Qiskit Aer (ノイズなし)

■第2段階評価: ノイズ環境下でのエラー抑制効果

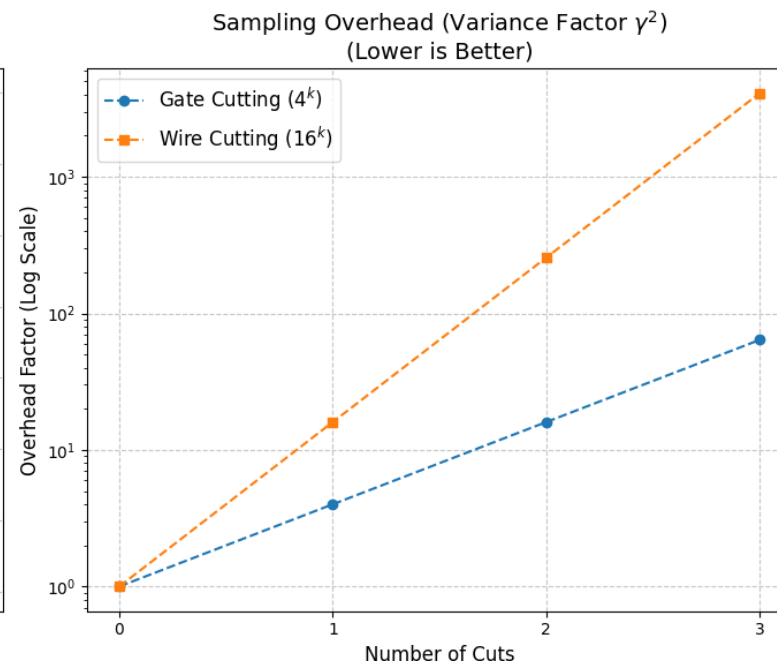
- 回路: クリフォード回路 (使用ゲートを限定「X/Y/Z/H/S/Sdg」、10量子ビット回路、深さ4)
- チップ: 超伝導型8x8格子チップ (阪大量子システム想定)
- 切断数: [0, 1, 2, 3, 4]箇所
- 実行ショット数: 各部分回路10000ショット固定
- シミュレータ: Stim (ノイズあり)

第1段階評価 (1/2)

まずノイズのない環境下で、切断数 k を変化させた際の復元精度を比較した。実行ショット数は固定としランダム回路を生成し行った。※切断位置は手法による復元精度の比較を平均的に行うため、MIPソルバを使わずにランダムに入れている。



左のグラフ（精度）では、同じショット数（各部分回路2000回）で実行した時、真の値からどれだけズレるかを示す。Wire Cuttingは係数の総和が大きいため、分散が大きくなり、同じショット数ではGate Cuttingより誤差が大きくなる。

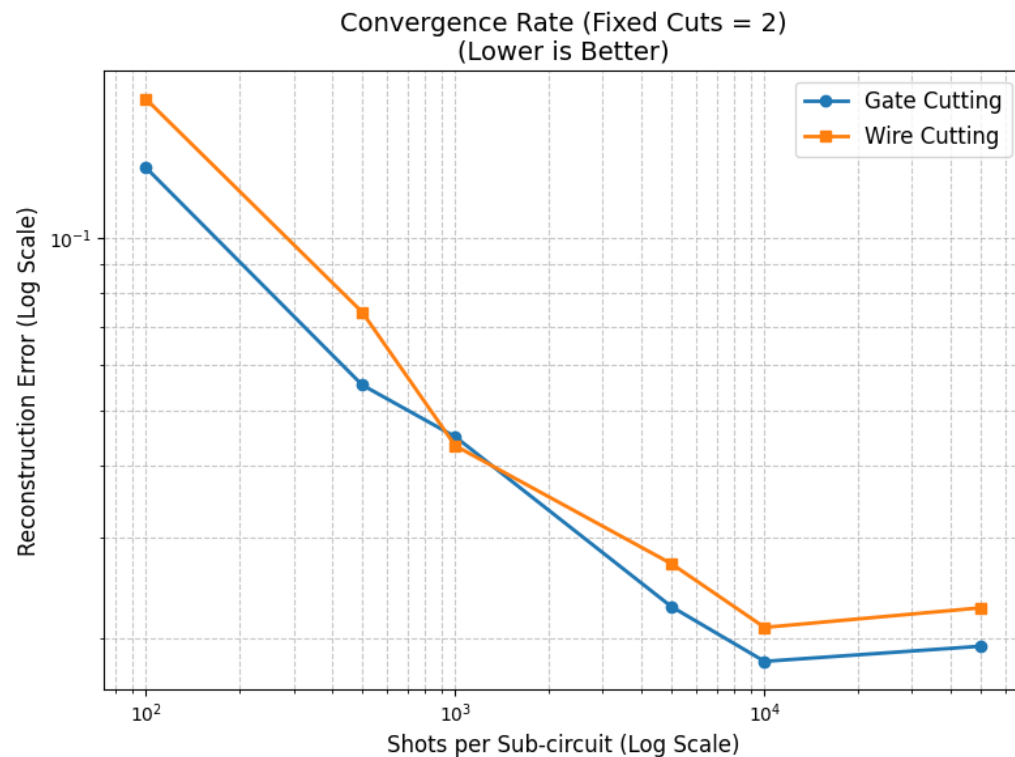


右のグラフ（オーバーヘッド）は、理論的なサンプリングコストの増大率（ γ^2 ）を対数グラフで示している。*Gate Cutting*は 4^k 、*Wire Cutting*は 16^k で増大するため、切断数が増えるほど*Wire Cutting*が圧倒的に不利である。

実験の結果、切断数 $k \geq 2$ において両手法も切断数に応じて推定値の分散が増大している。傾向としてGate Cuttingの方がWire Cuttingよりも分散が少ない。これは理論的なサンプリングコスト係数 4^k と 16^k によるものと思われる。このことよりGate Cuttingで問題ないことが示された。

第1段階評価 (2/2)

更に収束性の比較を行うため切断数を固定し、実行ショット数を変化させた際の復元精度を評価した。



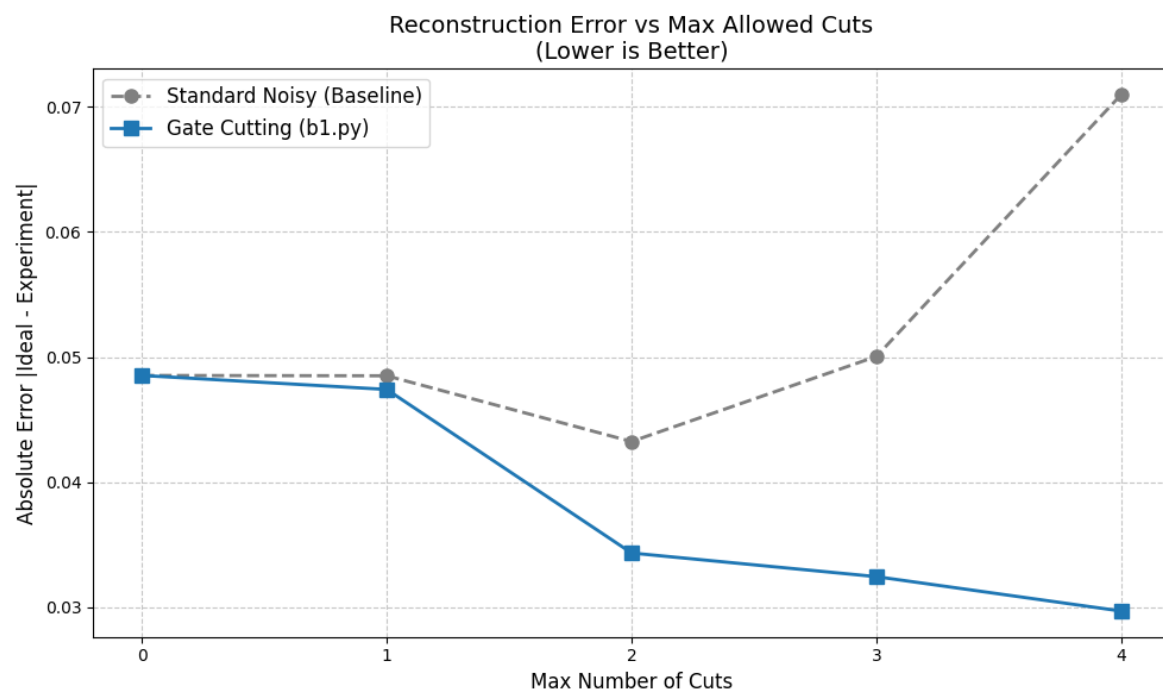
両手法ともにショット数の増加に伴い再構成誤差が減少する傾向（収束）が確認された。グラフ全体を通して一部を除き、Gate Cutting（青線）がWire Cutting（橙線）よりも下側に位置している。

クラウド量子計算機ではショット数が直接的な金銭的成本および実行時間に直結するため、より少ないショット数で高速に解へ収束するGate Cuttingの方が、実用上の優位性が極めて高いと結論付けられる。

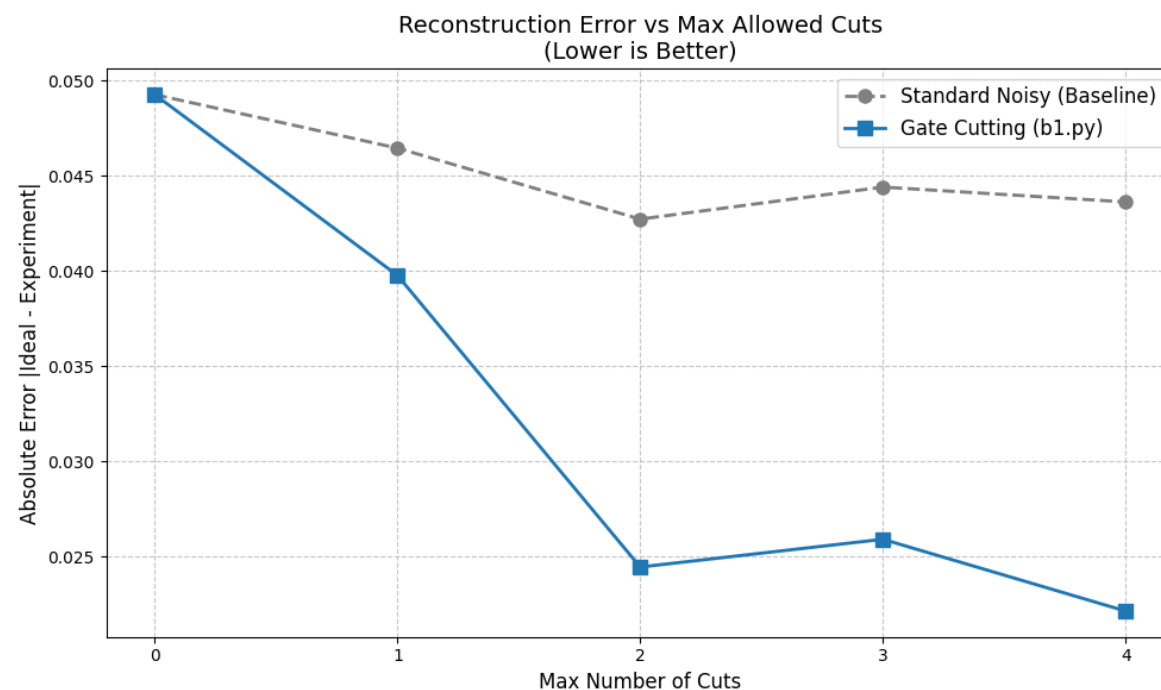
第2段階評価

次に、Stimを用いてノイズ環境下でのGate Cuttingの効果を検証した。左グラフは量子チップを固定して、ランダム回路を生成しMIPソルバで切断する箇所を増やしていった場合のエラー抑制効果を評価した図である。右グラフはチップもランダム化した結果である。

ランダム回路



ランダム回路 + ランダムチップ



切断数が増える毎にエラー抑制効果が向上していることが確認された。

結論

クラウド量子計算機におけるエラー抑制手法として、Gate CuttingとWire Cuttingの比較検証を行った。MIPソルバによる最適化と組み合わせることで、Gate Cuttingは現実的なオーバーヘッドで2量子ビットゲートのノイズを低減できることが示された。特に、Wire Cuttingと比較してサンプリングコストの増加を大幅に抑制できるため、NISQデバイスにおける高精度な量子計算を実現するための実用的なアプローチであると結論付けられる。

クラウドサービスへの適用と今後の課題

今後の課題

- OQTOPUSへの実装: オープンソースの量子コンピューティングフレームワークOQTOPUSに本手法を実装し、広く利用可能にする。
- 実機での検証: 実機での評価を行い、シミュレーション結果との比較を実施する。
- 実行時に手法を切り替える: 回路の特性やノイズ環境に応じて、Gate CuttingとWire Cuttingを動的に選択するアルゴリズムを開発する。
- MPIソルバの制約事項の改善
 - MIPやグラフの重み付けを改良する
 - 部分回路の大きさがバランスよくなるようにする
 - エンタングルメントの弱い箇所を切断するようにする
- 他手法の実装と評価: Gate CuttingやWire Cutting以外の回路切断手法を実装し、比較評価を行う。
- ベンチマークQASMでの評価: 既存のベンチマーク回路を用いて、提案手法の汎用性と効果を検証する。
- SWAPを減らす切断: CutQC のように、SWAPゲートを減らすことを目的とした切断手法の実装と評価を行う。事前トランスパイルの結果からSWAPゲートを含む部分回路を検出し、切断によってこれを回避するアプローチを検討する。
- クリフォード回路シミュレータを用いた事前評価: Stim 等のクリフォード回路シミュレータを用いて、より大規模な回路に対しても事前評価を行う手法を検討する。
- Virtual CXゲート: ユーザがvirtual CXゲートを指定できるようにし、MIPソルバでそのゲートを優先的に切断する機能を追加する。

OQTOPUS

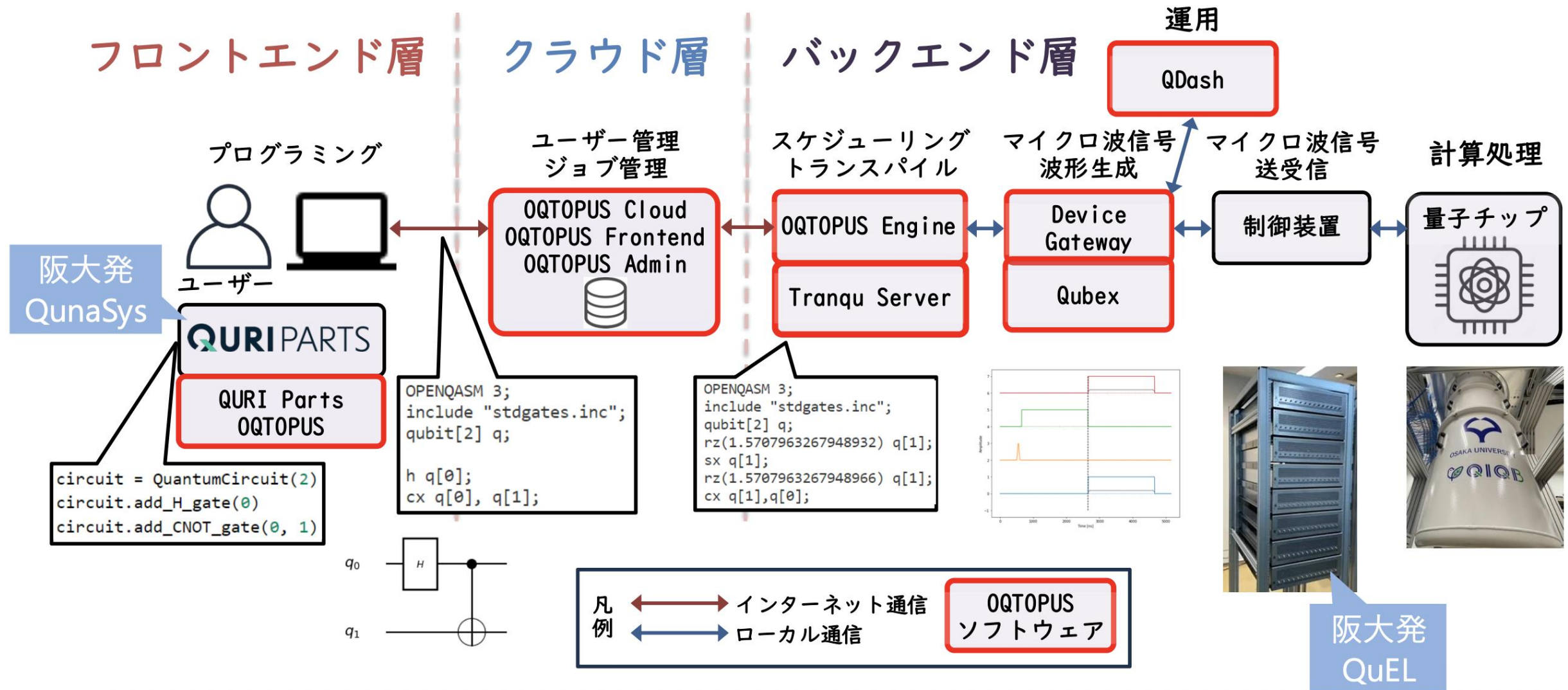
オープンソースの量子コンピュータの基本ソフトウェア「OQTOPUS」(オクトパス)に今後、搭載していきます。

OQTOPUSの名称の由来の一つ
大阪大学のスーパーコンピュータ
OCTOPUSへのリスペクトです！



<https://oqtopus-team.github.io/>

システム構成



謝辞

本研究は以下による助成を受けて行われました。

- 科学技術振興機構（JST）共創の場形成支援プログラム（COI-NEXT）「量子ソフトウェア研究拠点（研究代表者：北川勝浩） Grant No.JPMJPF2014」
- NEDO（国立研究開発法人新エネルギー・産業技術総合開発機構）の助成事業（JPNP25014）

Appendix

バックエンド層(2/3)

トランスパイラ

- ユーザが実装した量子回路のゲートを量子デバイスがサポートするゲートに変換

```
OPENQASM 3;
include "stdgates.inc";
qubit[2] q;

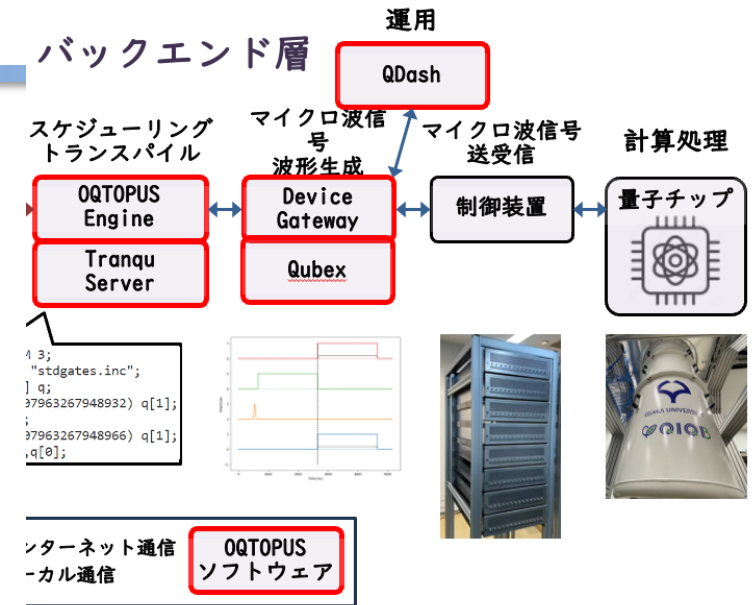
h q[0];
cx q[0], q[1];
```

変換

hは未サポート
rz, sx, rzに変換

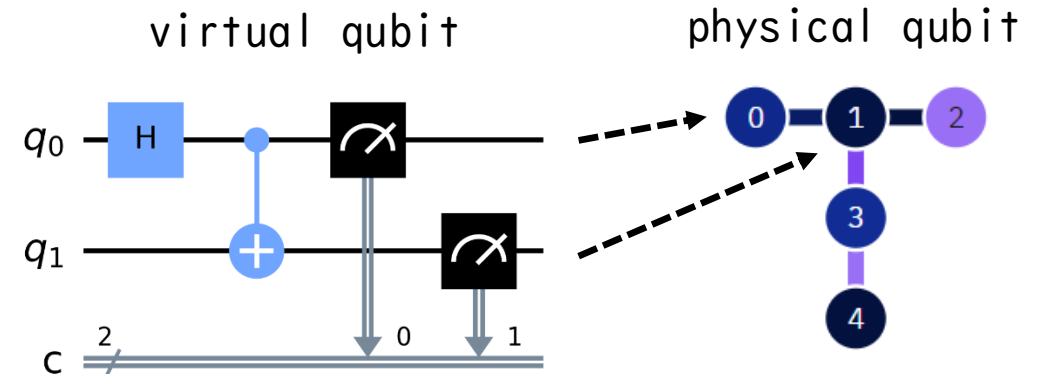
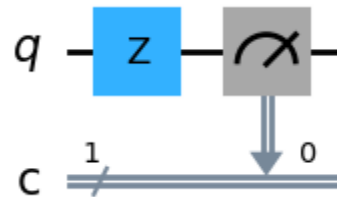
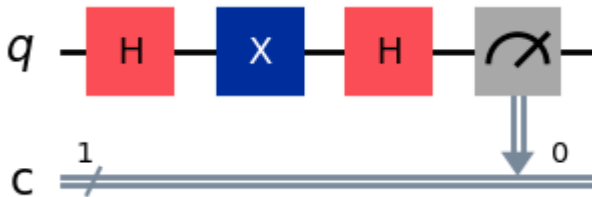
```
OPENQASM 3;
include "stdgates.inc";
qubit[2] q;

rz(1.5707963267948932) q[1];
sx q[1];
rz(1.5707963267948966) q[1];
cx q[1], q[0];
```



- 量子プログラムの量子ビット(virtual qubit)を量子デバイスの量子ビット(physical qubit)にマッピングする。量子ビット間の接続性を考慮
 - 最適なマッピングを求めるのはNP困難問題
 - ノイズの少ない場所を使いたい

- 短い量子回路に変換



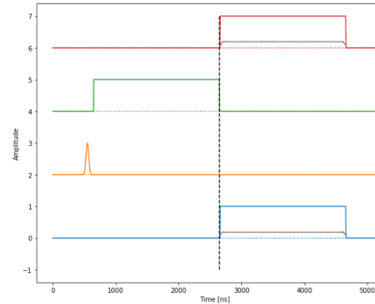
バックエンド層(3/3)

マイクロ波の波形生成

- 量子回路の情報を
マイクロ波のパルスシーケンスに変換

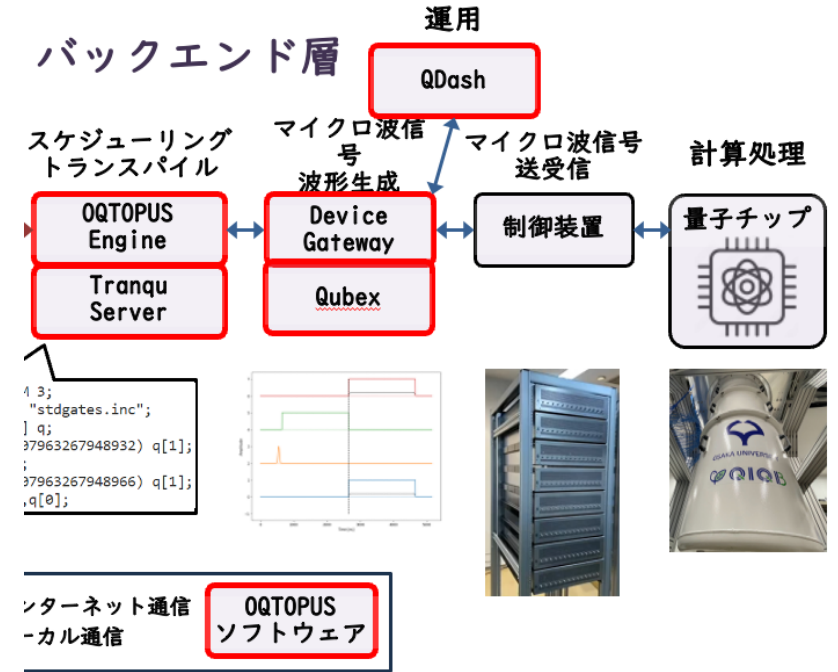
```
OPENQASM 3;  
include "stdgates.inc";  
qubit[2] q;  
rz(1.5707963267948932) q[1];  
sx q[1];  
rz(1.5707963267948966) q[1];  
cx q[1],q[0];
```

変換



マイクロ波の送受信

- 量子チップにマイクロ波を送受信。電子回路の知識が必要
- 大阪大, QuEL, e-Treesが開発したソフトウェアを利用
- 制御装置のプログラムはFPGA上で動作



キャリブレーション(1/2)

超伝導量子ビットの操作にはマイクロ波制御が必要である

▼超伝導量子ビット操作の概要

- 超伝導量子ビットはウェハ上に電氣的な回路を作成して、人工的に2準位系を構築している
- 量子ビットの操作はマイクロ波制御装置を介して行われている
- チップが完成したらすぐに量子ビットとして機能するわけではなく、設計値に応じたマイクロ波のキャリブレーションを行う必要がある
- チューニング対象は照射するマイクロ波の周波数や強度、パルス長など様々であり、複数の種類の実験を組み合わせで行われる

キャリブレーション(2/2)

Calibrationは複数の実験を組み合わせて行われており、量子ビットの数がスケールするにつれてトレースが困難になる

▼阪大環境の場合

- 1量子ビット Calibration
 - 64量子ビットの場合、各64ノードそれぞれについてCalibrationが必要
- 2量子ビット Calibration
 - 112のエッジそれぞれについてCalibrationが必要

